

Les bases d'Unix

(3^{ème} année de licence)

Filières fondamentale et professionnelle

Pr. M. JEDRA

Table des matières

Chapitre 1 : Présentation générale du système Unix

- 1.1 Historique
- 1.2 Structure du système
- 1.3 Normalisation

Chapitre 2 : Communication avec le système

- 2.1 Session utilisateur
- 2.2 Communication entre utilisateurs
- 2.3 Programmes d'information

Chapitre 3 : Le système de fichiers

- 3.1 Les fichiers Unix
- 3.2 Commandes de manipulation des fichiers
- 3.3 Commandes de manipulation des répertoires
- 3.4 Redirection des entrées/sorties
- 3.5 Les droits d'accès
- 3.6 Génération de noms

Chapitre 4 : L'éditeur pleine page vi

- 4.1 Les modes de travail de vi
- 4.2 Edition d'un fichier

Chapitre 5 : Gestion des processus

- 5.1 Le concept de processus
- 5.2 Appels système pour la création de processus
- 5.3 Gestion des processus
- 5.4 Droits d'accès étendus pour les processus

Chapitre 6 : Le shell

- 6.1 Notions et mécanismes de base
- 6.2 Les variables shell
- 6.3 Les shell scripts
- 6.4 Programmation shell

UNIX

Présentation générale

Unix est une marque déposée de AT & T

1.1 Historique:

1968: Fin du projet MULTICS,

Etude commune des Bell Labs et General Electric au MIT.

- Axes de recherche:
- mémoire virtuelle,
 - protection,
 - ordonnancement,
 - système de gestion de fichiers.

Bell Labs sont des labos de AT & T et Western Electric.

1970: Première version d'Unix

- système mono-utilisateur,
- système de gestion de fichiers,
- outils de traitement de textes,
- noyau de système élémentaire,
- interpréteur de commandes élémentaire.

Travail initialisé par Ken THOMPSON

1971: Première version d'Unix

+ documentation

+ plusieurs extensions

- système de fichiers,
- gestion de processus,
- interface système,
- utilitaires,
- transport sur PDP 11/20

Première version officielle interne Unix V1 signée:

Ken THOMPSON et Denis RITCHIE.

1972: Unix V1+ "pipes" = V2

Transport sur PDP 11/20/30/34/40/45/60/70.

1973: Unix ré-écrit en C

BCPL ==> B ==> NB ==> C (D. RITCHIE)

<Langages interprétés> : <Compilateur cc >

1974: Unix V5, distribuée gratuitement à des universités
(UC Berkley, Colombia U.)

1975: Unix V6, Première version distribuée pour 250 \$.

1977: Unix 32V sur Vax (32bits)
UC.Berkley (BSD) ==> C-Shell, éditeurs,...
BSD: Berkley Software Distribution

1979: Unix V7 = V6+portabilité
plusieurs implantations.

1980: "ONYX" première version pour micros

1981: Première version commercialisée par AT & T ==> System III
U.C.Berkley ==> BSD4.1
Deux produits indépendants

1983: AT et T ==> System V
U.C.Berkley ==> BSD4.2

1984: Microsoft ==> Xenix version pour PC 8086

1987: AT et T ==> System V Release 3
U.C.Berkley ==> BSD4.3
Santa Cruz Opérations ==> SCO Unix pour PC 80386

1990: AIX3.0 ==> Le système Unix d'IBM

1991: AT & T et SUN ==> System V Release 4
Intégration des fonctionnalités du système V et BSD
Standard (Posix)

Actuellement: AIX d'IBM, Solaris de SUN, Linux, ..., etc

1.2 Structure du système

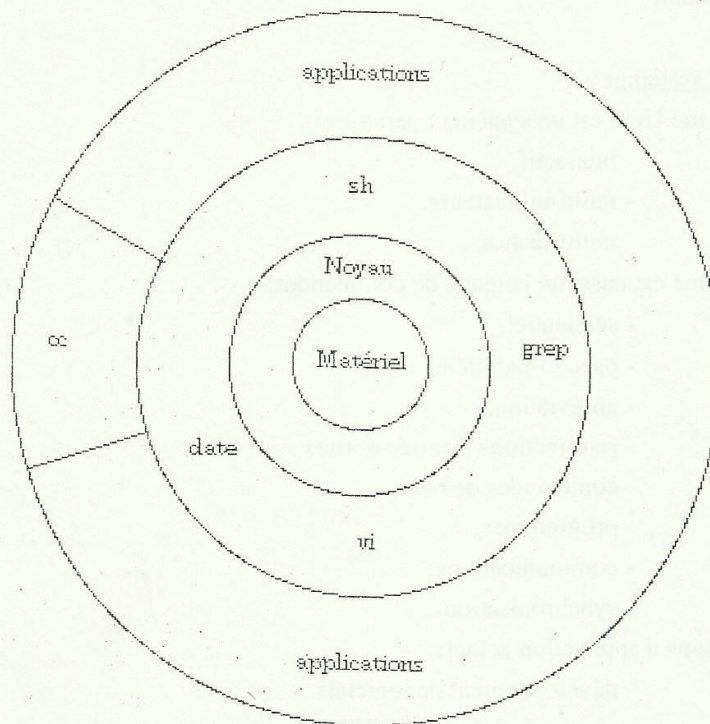


Fig 1.1 Structure du système

Le noyau :

- gère les ressources matérielles (mémoires, unités d'E/S...), les fichiers, l'allocation du temps CPU.
- gère les processus.

Le shell :

Le shell (coquille) est un utilitaire qui sert d'intermédiaire entre la saisie des commandes et le noyau, c'est un interpréteur de commandes. Il existe plusieurs shell :

sh	Bourne-shell (standard)	Steve Bourne 1970.
csh	C-shell	(Berkley)
ksh	Korn-shell	David Korn
bach	Bourne-again-shell (Linux)	

...

Le ksh est une extension du Bourne-shell qui intègre les caractéristiques principales du C-shell.

La couche des outils et des applications:

C'est l'ensemble des programmes et des utilitaires qui ont été écrits petit à petit pour améliorer le système.

Les qualités du système:

Le système Unix est un système opératoire:

- interactif,
- multi-utilisateurs,
- multi-tâches,

Le système est aussi un langage de commandes:

- séquentiel,
- pseudo-parallèle,
- abréviations,
- re-directions d'entrée-sorties
- commandes de base,
- programmes,
- communications,
- synchronisation...

Les champs d'application actuels:

- développement de logiciels
- applications industrielles (extensions temps-réel)
- communications (courrier électronique, transfert de fichiers,...etc)

Le système évolue maintenant vers la distribution et les systèmes répartis

- système de gestion de fichiers répartis (NFS de Sun),
- exécution de commandes à distance,
- extension des mécanismes de nommage,
- noyau à commutation de messages,
- communications inter-processus,

L'environnement de programmation:

- langages: C, C++, Pascal, Fortran, Lisp, APL, Prolog, JAVA,...,
- éditeurs de texte ligne (ed), écran (vi, emacs,...etc),
- utilitaires de traitement de texte (TROFF, NROFF),
- communications inter-utilisateurs (mail),
- outils graphiques de base (GKS, PHIGS),
- interfaces utilisateur multi-fenêtres (Xwindows, Sunview, KDE ...,etc),
- outils de gestion de logiciels (MAKE, SCCS),
- outils pour Internet (TCP/IP, DNS, POP...etc)
- service d'authentification Client-Serveur (Kerberos)
- applications...

1.3 Normalisation

Le but de la normalisation est de définir les interfaces avec le système et de laisser libre la mise en oeuvre du noyau (structure, optimisation,...,etc).

Les organismes internationaux de normalisation:

ISO : International Standards Organisation

JTC1=Join Technical Comitee (comité technique dédié à l'informatique)

Les organismes nationaux:

ANSI (USA), BSI (UK), DIN (RFA), AFNOR (F),...,etc

Les sociétés de service informatique:

Les SSI font des propositions de normes, et font des choix pour rendre les normes opérationnelles.

/usr/group	USA	"USR/group standards" 1984
------------	-----	----------------------------

IEEE	USA	"POSIX: Portable Open System Interface"
------	-----	---

X-OPEN	EU	"Portability Guide"
--------	----	---------------------

Les sociétés:

Imposent de nouveaux standards

AT & T (SIVD = System V Interface Definition)

AT & T (SVVD = System V Vérification Suite)

Les groupements:

Unix International : AT & T, SUN

OSF: IBM, HP, DEC, BULL,...,etc

IEEE DP12 du comité 1003 chargé de POSIX

Le comité 1003 se compose de sous-comités:

1003.1: Norme de base.

1003.2: Langages de commande (shell).

1003.3: Tests et suite de vérification de conformité.

1003.4: Aspect temps réel.

1003.5: POSIX et ADA

1003.6: Sécurité à partir de la classification de l'Orange Book du DOD.

Cette norme POSIX assure la portabilité du système indépendamment du langage C.

X-OPEN groupement de constructeurs

- Des constructeurs européens (BULL, ICL, Olivetti, Siemens-Nixdorf...)

- Quelques constructeurs Américains dont AT & T

Le but de X-OPEN est de proposer, d'influencer et d'adopter des normes de manière cohérente dans un parc hétérogène.

UNIX

Communication avec le système

2.1 Session utilisateur

```
login: nom-utilisateur < rc >
password: xxxxxx < rc >          # mot de passe non affichée.
```

Si soit le nom-utilisateur ou le mot de passe est incorrect, le système répondra par:

```
login incorrect
login:
suivi de password:
```

Si votre **login** est correct automatiquement il y a apparition du prompt du shell :

```
$      avec le Bourne shell
ou    %      avec le C-shell
ou un autre prompt choisi par l'administrateur du système.
```

Pour changer le mot de passe on utilise la commande **passwd**

```
$passwd
Changing password
Old password: [Entrer l'ancien mot de passe]
New password: [Entrer le nouveau mot de passe]
Re-enter new password: [Entrer le nouveau mot de passe]
```

On signale que des fichiers prédéfinis contenant des commandes sont exécutés au début et à la fin d'une session, par exemple : des commandes de configuration du terminal, de lancement d'une application,...,etc.

```
Pour BSD / C-shell : .login et .logout
Pour AT & T / shell (Bourne) : .profile
```

Pour sortir de la session on utilise :

```
$logout < rc >          # version BSD
< Ctrl +D >             # version AT & T.
$exit <rc>              # Linux
```

2.2 Communication entre utilisateurs

La communication entre utilisateurs peut se faire en ligne ou en différé.

Communiquer en ligne (**write** et **wall**)

- communication immédiate avec les autres utilisateurs "logués".
- une réponse immédiate est possible

Communiquer en différé (**mail**)

- le message est permanent, il est sauvé dans un fichier.
- différentes possibilités lors de l'obtention d'un message

wall envoie un message à tous les utilisateurs. L'appel de cette commande suppose des droits d'accès adéquats (administrateur). Le plus souvent elle est dans le répertoire /etc.

Exemple :

\$wall

Le système sera arrêté dans 10 minutes.

Salutations Administrateur

\$

Le message apparaît sur tous les terminaux des utilisateurs "logués".

write permet d'envoyer des messages en ligne à un autre utilisateur connecté.

\$write utilisateur [Terminal]

message

.....

Le terminal est spécifié dans le cas où l'utilisateur est connecté à travers plusieurs terminaux. Dans le message reçu par le destinataire, se trouvent les informations suivantes :

- le nom de l'expéditeur,
- le terminal d'émission,
- l'heure de début de l'émission,
-etc.

Exemple :

Terminal tty01 (Youssef)

\$write Karim

Message from Karim tty02.....

Bonjour Karim

Veux tu aller au cinéma ce soir

Oui

OK

<Ctrl+D>

\$

Terminal tty02 (Karim)

Message from Youssef tty01.....

write Youssef

Bonjour Karim

Oui

OK

EOT

<Ctrl+D>

\$

<Ctrl+D> permet de sortir de **write**

Pour envoyer un message à un utilisateur, celui-ci doit en donner les droits avec la commande de contrôle de réception de message **mesg**

\$mesg [y ou n]

Elle permet à l'utilisateur de verrouiller son écran face aux éventuels messages.

y (oui) pour annuler le verrouillage **n** (non) pour établir le verrouillage

mail permet d'échanger du courrier avec d'autres utilisateurs. En général les messages arrivés atterrissent dans un fichier du répertoire /usr/mail (ou usr/spool/mail). Une fois les messages lus ils sont soit stockés dans des fichiers en local ou détruits. Par défaut, il s'agit du fichier mbox, dans le répertoire courant de l'utilisateur.

```
$mail                                # entrée en mode commande,
message1                            # les messages sont visualisés l'un après l'autre.
? <rc>                             # le prompt est généralement ? ou &
message2
?
```

Après le prompt ? on peut utiliser l'une des commandes suivantes :

```
d      supprimer le message et afficher le suivant
s [fichier]  sauvegarder le message dans mbox, ou dans [fichier] si spécifié.
-      message précédent.
r      réponse à l'expéditeur du message lu
w      identique à s mais sans entête
h      liste les entêtes des messages
m [utilisateurs] renvoie du message aux [utilisateurs]
q      sortie de mail avec sauvegarde dans mbox
x      sortie sans sauvegarde
?      aide en ligne
```

```
$mail [adresse]
Subject:      #Sujet du message
Cc:           #Copie conforme
message
« . » ou <Ctrl+D>
```

Exemples

```
$mail Ahmed                        # adresse dans la même machine
Subject: Sport
Cc: Youssef
Il y aura une réunion concernant le sport universitaire
le Lundi 09/10/02 à 18h à la salle des séminaires.
Salutations Karim
.
```

```
$mail
From Karim Thu Oct 02 16 :43 :27.....
.....
Il y aura une réunion concernant le sport universitaire
le Lundi 09/10/02 à 18h à la salle des séminaires.
Salutations Karim
?
```

```
$mail Ahmed < message          # envoie d'un message sous forme de fichier
```

```
$mail Ahmed@fsr.ac.ma < message # envoie d'un message à un utilisateur sur
                                # une autre machine
```


2.3 Programmes d'information

Ce sont des commandes qui servent à l'utilisateur pour obtenir des informations sur l'état de son système Unix. Il s'agit, entre autres, de :

date commande qui affiche la date et l'heure du système.

```
$date [Format]
$date mmddHHMM[yy]      # mm le mois dd le jour
                        # HH l'heure MM les minutes
                        # yy l'année
```

who commande qui indique les utilisateurs connectés au système.

```
$who [option]
$who am I
```

Cette commande sans paramètres affiche pour tous les utilisateurs connectés les informations suivantes :

- nom de l'utilisateur,
- terminal sur lequel il travaille,
- heure de connexion,
-, etc

cal affiche le calendrier

```
$cal [[mois] année]
```

man affiche la documentation intégrée concernant une commande.

```
$man commande
```

Le système Unix possède une documentation intégrée de toutes les commandes, c'est une aide (Help) mis à la disposition de l'utilisateur.

UNIX

Le système de fichiers

3.1 Les fichiers Unix

Fichier Unix

C'est un ensemble d'informations associées à un nom. Il est considéré comme une suite d'octets (caractère ASCII ou code machine) stockés sur disque ou bande magnétique. Il n'y a pas d'organisation spéciale (séquentielle, indexée,...)

Les types de fichiers Unix :

- Fichiers ordinaires (segments):

Un fichier ordinaire est une séquence de caractères (sources, binaires, textes). Pas de structure imposée par le système; la structure logique des fichiers (enregistrements) est fixée par les programmes d'applications.

- Répertoires (directories) :

Un répertoire est un catalogue qui contient les noms des fichiers et des sous répertoires.

- Fichiers spéciaux (unités d'E/S) :

Ces fichiers représentent des interfaces avec les périphériques gérés par le système d'exploitation.

Organisation utilisateur du système de fichiers :

C'est une structure arborescente qui possède un répertoire racine et dont les noeuds sont des répertoires. Chaque répertoire peut contenir des fichiers et d'autres répertoires.

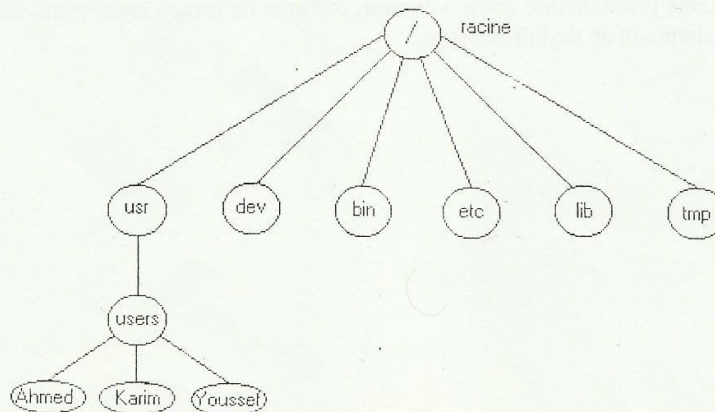


Fig 3.1 Schéma simplifié du système de fichiers

`usr` contient les répertoires utilisateurs (`home` pour Linux).

`lib` contient les bibliothèques (par exemple `lib.c`)

`dev` contient les fichiers spéciaux : `tty01`, `lp01`, `dk01`, `fl01`,....etc

bin contient les commandes : **date**, **cat**, **who**,...,etc
etc contient les fichiers système : **passwd**, **group**, **hosts**,...,etc
tmp contient les fichiers temporaires du système.

Tout fichier est nommé par son chemin d'accès (pathname) dans l'arborescence.

/usr/users/Ahmed/mbox est la boîte aux lettres de l'utilisateur Ahmed.

/usr/users/Ahmed est le répertoire personnel (Home directory) de l'utilisateur Ahmed.

Conventions de nommage

.. répertoire parent
.
/ racine de l'arborescence.

Le nom d'un répertoire ou d'un fichier peut contenir des chiffres et des lettres (minuscules ou majuscules), ainsi que des caractères spéciaux dont l'utilisation est déconseillée.

Organisation physique

Le disque dur est un ensemble de plateaux empilés verticalement sur un même axe. Chaque plateau est composé d'une ou de deux faces, divisées en pistes. Chaque piste est elle-même divisée en secteurs de taille fixe entre 512 à 4096 octets. Le secteur constitue la plus petite unité de lecture/écriture sur le disque. L'opération qui permet d'organiser un disque vierge en un ensemble de pistes et de secteurs s'appelle formatage physique.

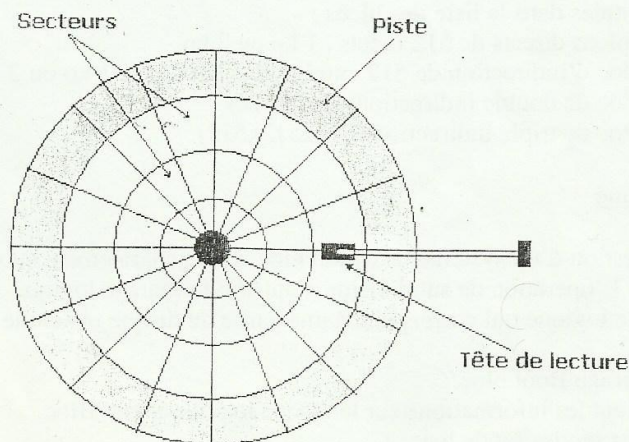


Fig. 3.2 Organisation d'un disque dur

Le système d'exploitation utilise pour les échanges entre la mémoire centrale et le disque dur des blocs et non des secteurs. Un bloc est constitué de plusieurs secteurs. Les méthodes d'allocation des blocs aux fichiers sont nombreuses mais la technique la plus utilisée est l'allocation indexée.

Allocation indexée

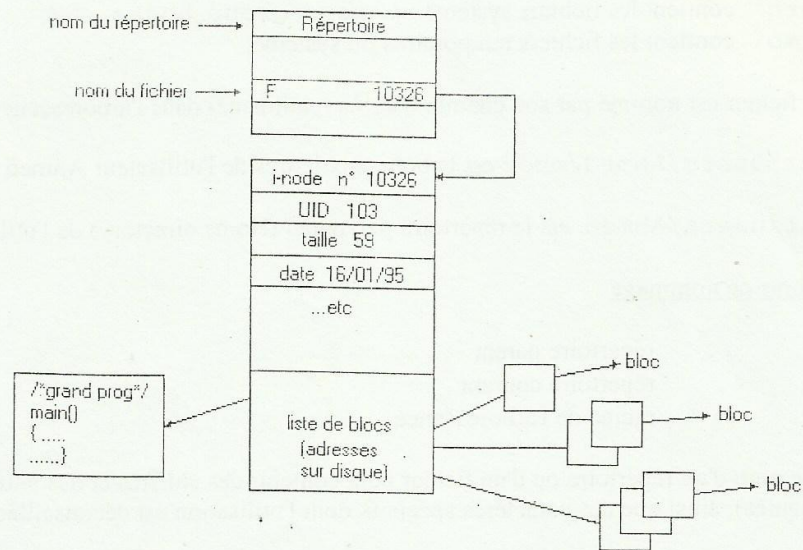


Fig. 3.3 Organisation interne d'un fichier

Un fichier : i_node -----> numéro de bloc

Un bloc (512, 1024 ou 2048 octets): taille, UID, liens, date, liste de blocs utilisés...etc

Indirections -----> arbre de blocs

Il y a 13 entrées dans la liste des blocs :

10 blocs directs de 512 octets , 1 ko ou 2 ko.

1 bloc d'indirection de 512 entrées de 512 octets, 1 ko ou 2 ko.

1 bloc de double indirection (512x512)

1 bloc de triple indirection (512x512x512)

Organisation logique

Pour faciliter la gestion d'un système d'exploitation il est généralement subdivisé en morceaux appelés partitions. L'opération de subdivision s'appelle formatage logique. Chaque partition constitue un disque logique qui correspond à une partie du disque physique et elle comporte :

- un bloc de démarrage Boot bloc,
- un bloc qui contient les informations sur le disque logique Super Bloc,
- une table pour les inodes Inode list,
- des blocs de données.

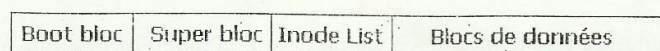


Fig. 3.4 Structure d'une partition

3.2 Commandes de manipulation de fichiers

`$ls [options] [nom de répertoire]`

La commande `ls` affiche les noms des fichiers et des répertoires du répertoire indiqué par son nom (chemin). Si le nom du répertoire est omis c'est le contenu du répertoire courant qui est affiché.

[options]

-l + les droits d'accès
-c en colonnes
-a les fichiers commencent par . (cachés).
-d les répertoires
-i + les numéros i-node.
..... etc

Les options de `ls` peuvent être mixées par exemple : `$ls - ail`

<code>\$cat F G</code>	affichage du contenu des fichiers <code>F</code> et <code>G</code>
<code>\$cp F/G</code>	copie de <code>F</code> sur <code>G</code> .
<code>\$rm F</code>	destruction de <code>F</code> .
<code>\$mv F G</code>	re-nommage de <code>F</code> en <code>G</code> ou déplacement de <code>F</code> dans <code>G</code> si <code>G</code> est un répertoire.
<code>\$more F</code>	équivalent de <code>cat</code> mais page par page.
<code>\$ln F G</code>	deux noms pour un même fichier (même numéro d'i-node).
<code>\$pr F</code>	impression paginée de <code>F</code> chaque page débute par une entête contenant : <date> <heure> <nom> <numéro de page>
<code>\$lp F</code>	impression d'un fichier <code>F</code>

3.3 Commandes de manipulation des répertoires.

<code>\$pwd</code>	impression du répertoire courant dans l'arborescence.
<code>\$cd <nom_rep></code>	changement de répertoire courant.
<code>\$mkdir <nom_rep></code>	création d'un sous répertoire.
<code>\$rmdir <nom_rep></code>	destruction d'un répertoire s'il est vide.

3.4 Redirections des entrées/sorties

Les entrées/sorties standards.

Quand un utilisateur se connecte à Unix, trois fichiers sont ouverts :

- entrée standard (**stdin**) : le clavier où sont entrées les commandes.
- sortie standard (**stdout**) : l'écran où sont affichées les résultats des commandes.
- sortie d'erreur (**stderr**) : l'écran où sont affichées les erreurs éventuelles.

Un nombre (le descripteur de fichier) est affecté à chaque fichier ouvert,

```
0 : stdin
1 : stdout
2 : stderr
```

Certaines commandes n'écrivent pas le résultat de leur action sur la sortie standard, mais dans un fichier, par exemple la commande **mail**.

Certaines commandes ne lisent pas sur l'entrée standard, mais sur un ou des fichiers. par exemple la commande **cat**.

Redirection des entrées/sorties

Le shell est en mesure de rediriger les résultats de l'action d'une commande vers un fichier différent de la sortie standard (écran). Il peut aussi lire les données d'un fichier différent de l'entrée standard (clavier) pour une commande qui attend les données du clavier.

- < redirection de l'entrée standard
- > redirection de la sortie standard (création)
- >> redirection de la sortie standard (ajout)
- << redirection spéciale de l'entrée standard (Herescripts).
- 2> redirection de la sortie d'erreurs (création)
- 2>> redirection de la sortie d'erreurs (ajout à un fichier existant)

Exemples :

```
$ls
Etudiant
Professeur
Spline.cpp
$ls > F
$ls
Etudiant
Professeur
Spline.cpp
F
$cat F
Etudiant
professeur
Spline.cpp
$
```

```
$who > Connecte
$cat Connecte
Ahmed tty01 Mai 12..... 10:18
Youssef tty02 Mai 12..... 08:45
$date >> Connecte
$cat Connecte
Ahmed tty01 Mai 12..... 10:18
Youssef tty02 Mai 12..... 08:45
12 Mai 1995 12:15:34
$
```


\$wc << fin

a b c d

e f g h

fin

2 8 16

\$

`/etc/passwd` contient les informations suivantes pour chaque utilisateur :

- nom de login
- mot de passe crypté
- numéro unique d'utilisateur : UID
- numéro unique de groupe : GID
- nom complet de l'utilisateur
- répertoire initial
- interpréteur de commande

`/etc/group` contient les informations sur le groupe auquel appartient l'utilisateur :

- nom du groupe
- numéro unique de groupe
- liste des utilisateurs de groupe

- fichier spécial en mode bloc (périphérique en mode bloc : disque)

```
brw-rw-rw-    1    root    ...    7    <date>    /dev/dk
```

- fichier spécial mode caractère (périphérique en mode caractère : transmission)

```
crw-rw-rw-    1    Ahmed    ...    214    <date>    /dev/tty1
```

- répertoire

```
drwxr-x--x    2    Ahmed    ...    593    <date>    nom_directory
```

Pour accéder à une feuille ou à un noeud dans une arborescence de répertoire, il faut avoir le droit en x sur tous les répertoires de niveau supérieur (c.à.d appartenant au chemin d'accès).

Les restrictions d'accès s'appliquent aux utilisateurs, un seul est exempt des contrôles d'accès : le super_utilisateur qui a pour login : **root** et le **UID=0**

Changement de propriétaire et de groupe

Ahmed un utilisateur propriétaire de F (ou administrateur) il peut changer le propriétaire F à l'aide de la commande **chown**

Exemple:

```
$chmod 664 F
```

```
600 ----> droit d'accès du propriétaire  rw-
60 ----> droit d'accès du groupe    rw-
4 ----> droit d'accès des autres    r--
```

Droits d'accès par défaut

Dans le système Unix les droits d'accès maximales pour les fichiers normaux et les répertoires sont :

```
rw-rw-rw-rwx (777) pour les répertoires
rw-rw-rw- (666) pour les fichiers
```

La commande **umask** permet de positionner les droits d'accès des fichiers par défaut.

Si aucun masque n'est positionné les droits par défaut sont:

```
Les éditeurs ----> -rw-r--r-- (644)
mkdir ----> drwxr-xr-x (755)
```

Exemple:

```
$umask 022
```

```
022 en octal: 000 010 010
ou exclusif 111 111 111 (777)
111 101 101 (755) ----> drwxr-xr-x pour les répertoires
```

```
022 en octal: 000 010 010
ou exclusif 110 110 110 (666)
110 100 100 (644) ----> -rw-r--r-- pour les fichiers
(éliminer les x)
```

drwxr-xr-x et **-rw-r--r--** sont devenus les valeurs par défaut.

3.6 Génération de noms (Jockers)

Le shell possède des caractères spéciaux pour définir des critères de recherche pour les noms de fichiers :

- * remplace une chaîne de longueur variable, même vide.
- ? remplacé un caractère unique quelconque.
- [. . .] représente une série ou une plage de caractères.

\$ls	a*	#liste des noms des fichiers commençant par a.
\$ls	[aA]*	#liste des noms des fichiers commençant par a ou A
\$ls	?*	#liste des noms de fichiers dont le nom est d'au moins un caractère
\$ls	a??	#liste des noms des fichiers ne contenant que 3 caractères dont la première est a

Pour annuler les fonction

UNIX

L'éditeur pleine page vi

4.1 Les modes de travail de vi

vi est un éditeur pleine page développé par l'Université de Californie Berkeley. **vi** est basé sur un éditeur "ligne" sous-jacent **ex** qui lui-même intègre l'éditeur ligne d'Unix **ed**. Il travaille selon deux modes:

- **Mode commande**, dans lequel toutes les saisies, lettres, touches et combinaisons de touches, représentent des commandes. En général, ces commandes ne terminent pas par la touche Entrée.

- **Mode saisie**, dans lequel toutes les saisies sont enregistrées.

passage en mode ex

<rc>

Le passage en mode ex est effectué par le bouton "EX" situé sur le panneau de commande. Le bouton "EX" est un bouton à pous-
sion qui permet de passer du mode "NORM" au mode "EX". Le mode "EX" est un mode de fonctionnement qui permet de passer en mode "EX" par le bouton "EX". Le mode "EX" est un mode de fonctionnement qui permet de passer en mode "EX" par le bouton "EX".

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Le passage en mode ex

Corrections

9.26

18.00

18.00

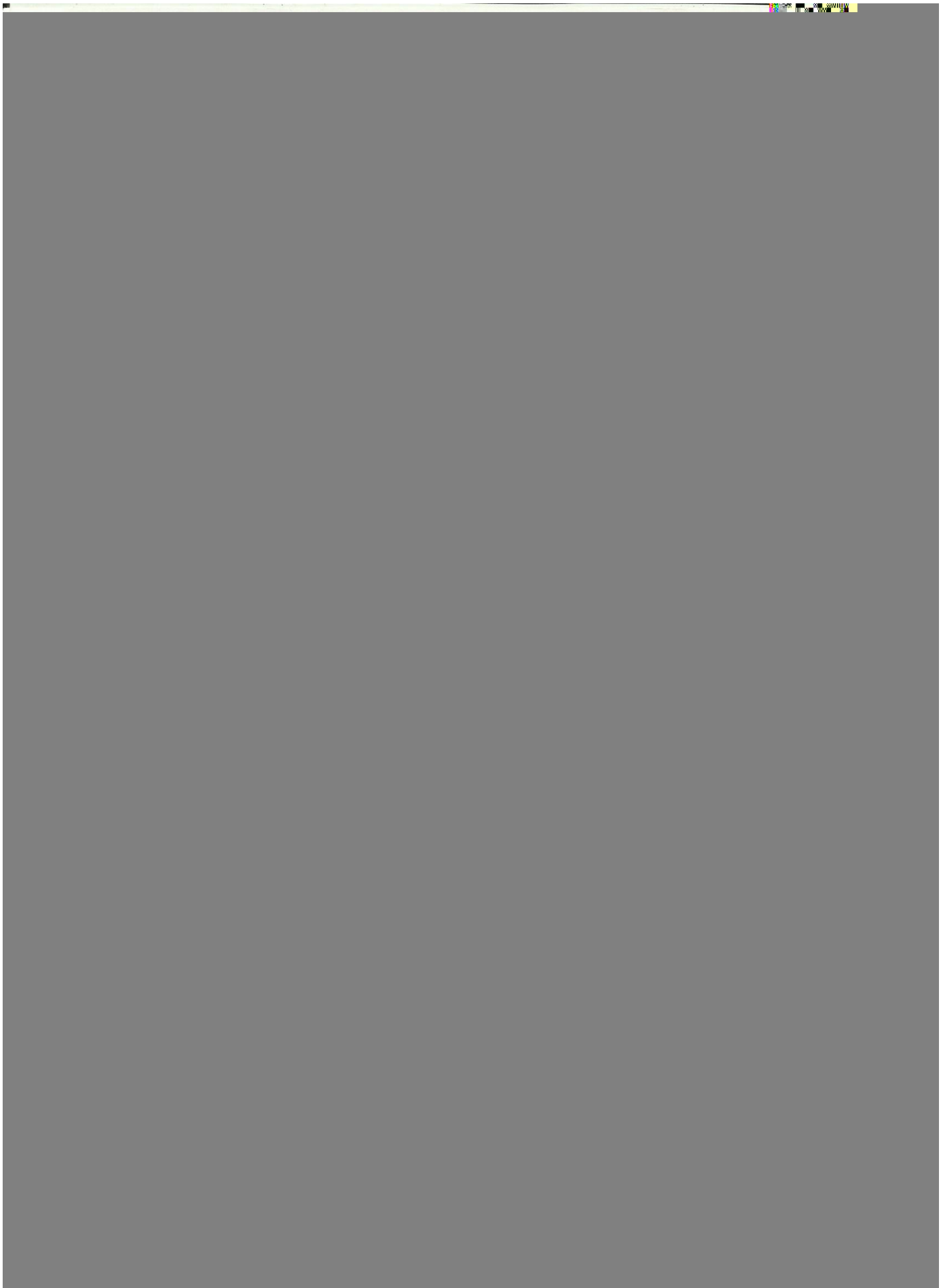
18.00

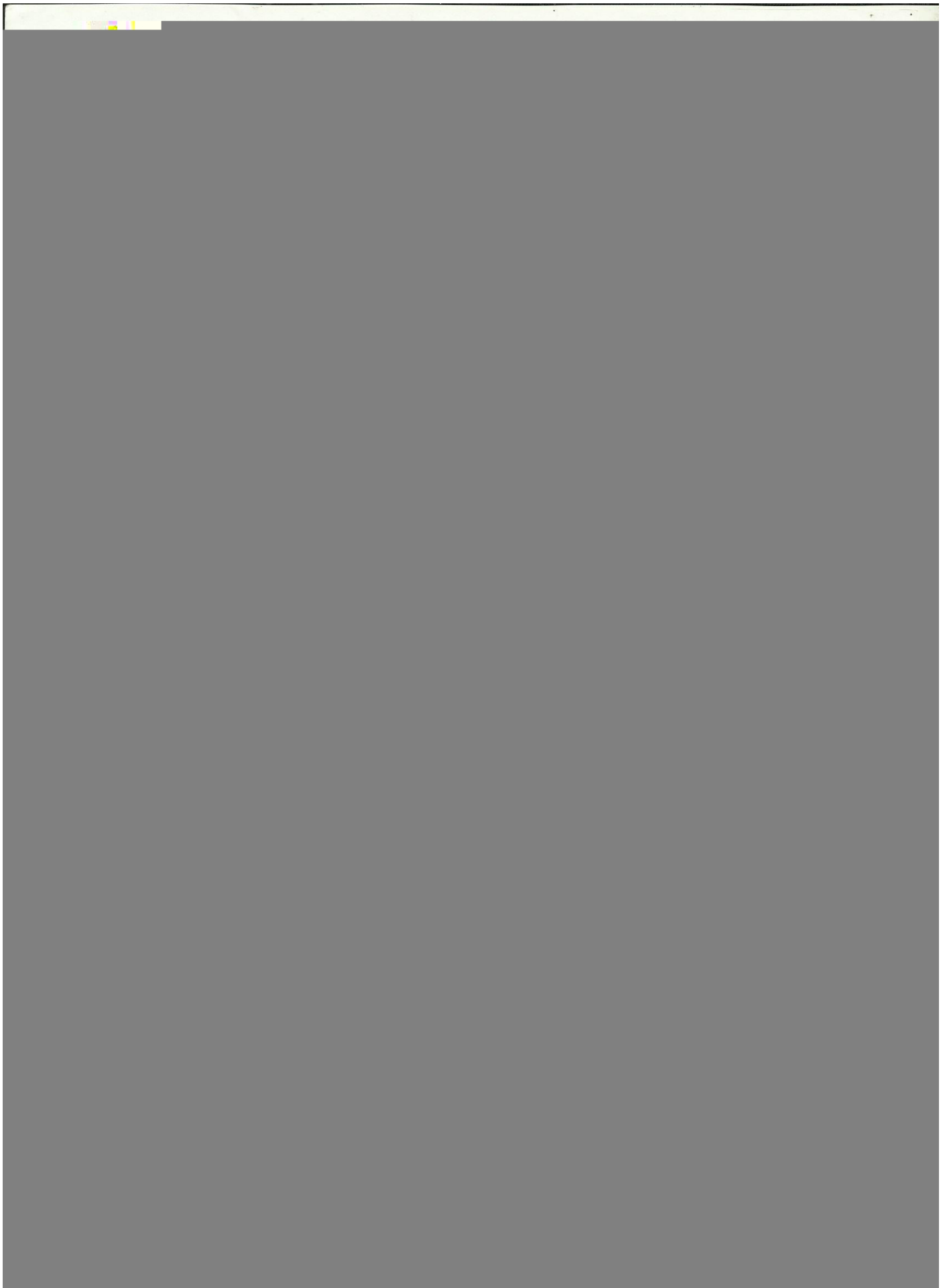
18.00

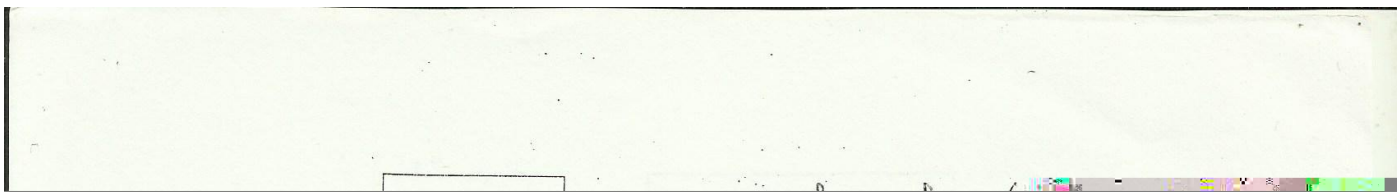
18.00

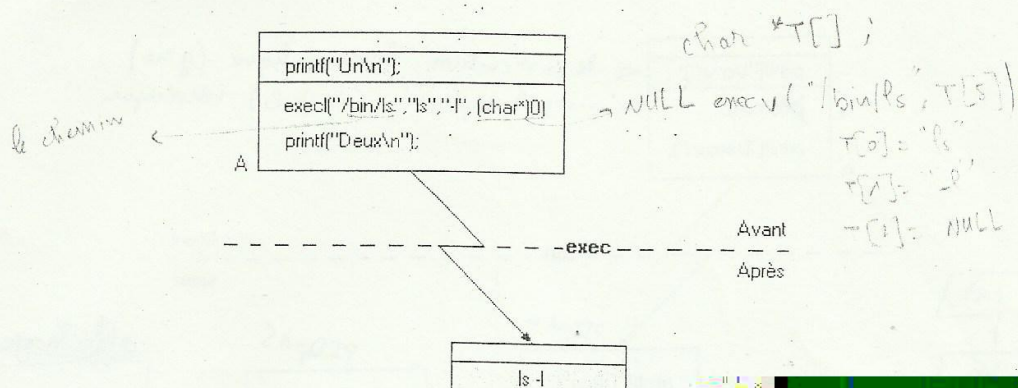
18.00

18.00









L'appel système `exit`

L'appel système `exit` est utilisé pour finir un processus. Il ne retourne pas de valeur et effectue les actions suivantes lors de son appel.

Communication entre processus

Les processus communiquent entre eux à l'aide de signaux ou à travers des pipes.

```
#include <stdio.h>
int pipe (int filedesc[2]) ;
```

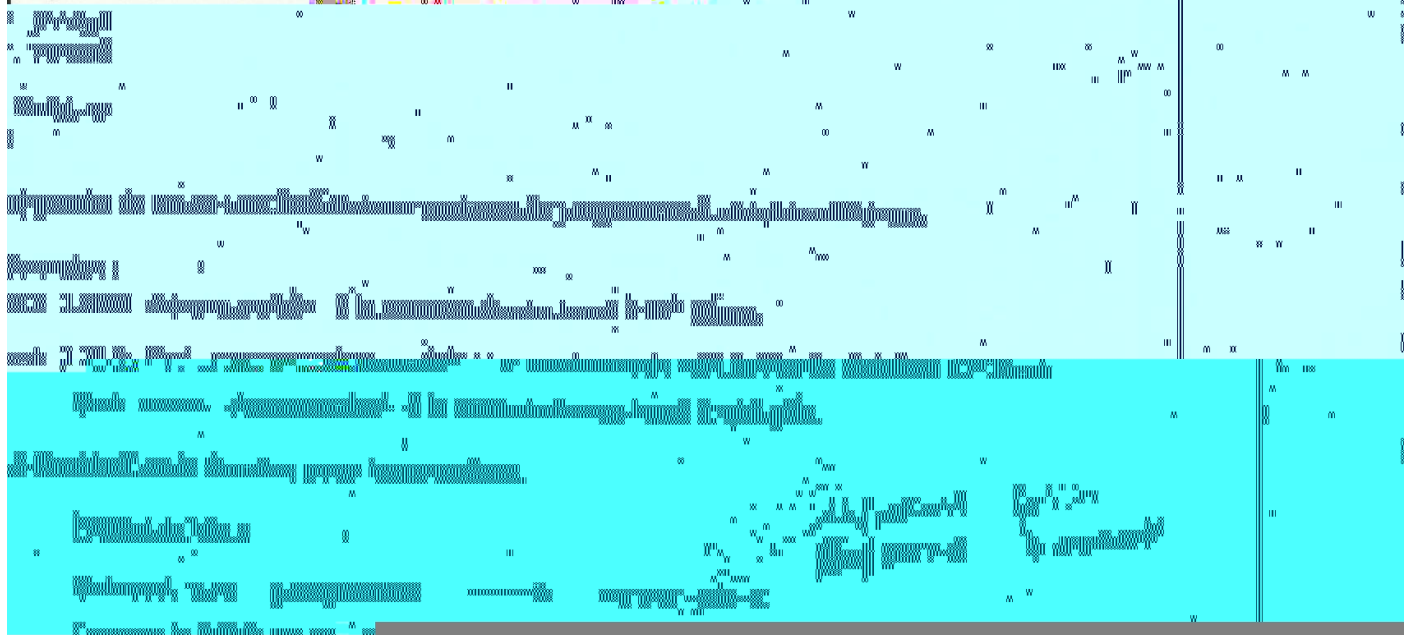
L'appel système `pipe` utilise deux descripteurs

Commendes de casti. 1

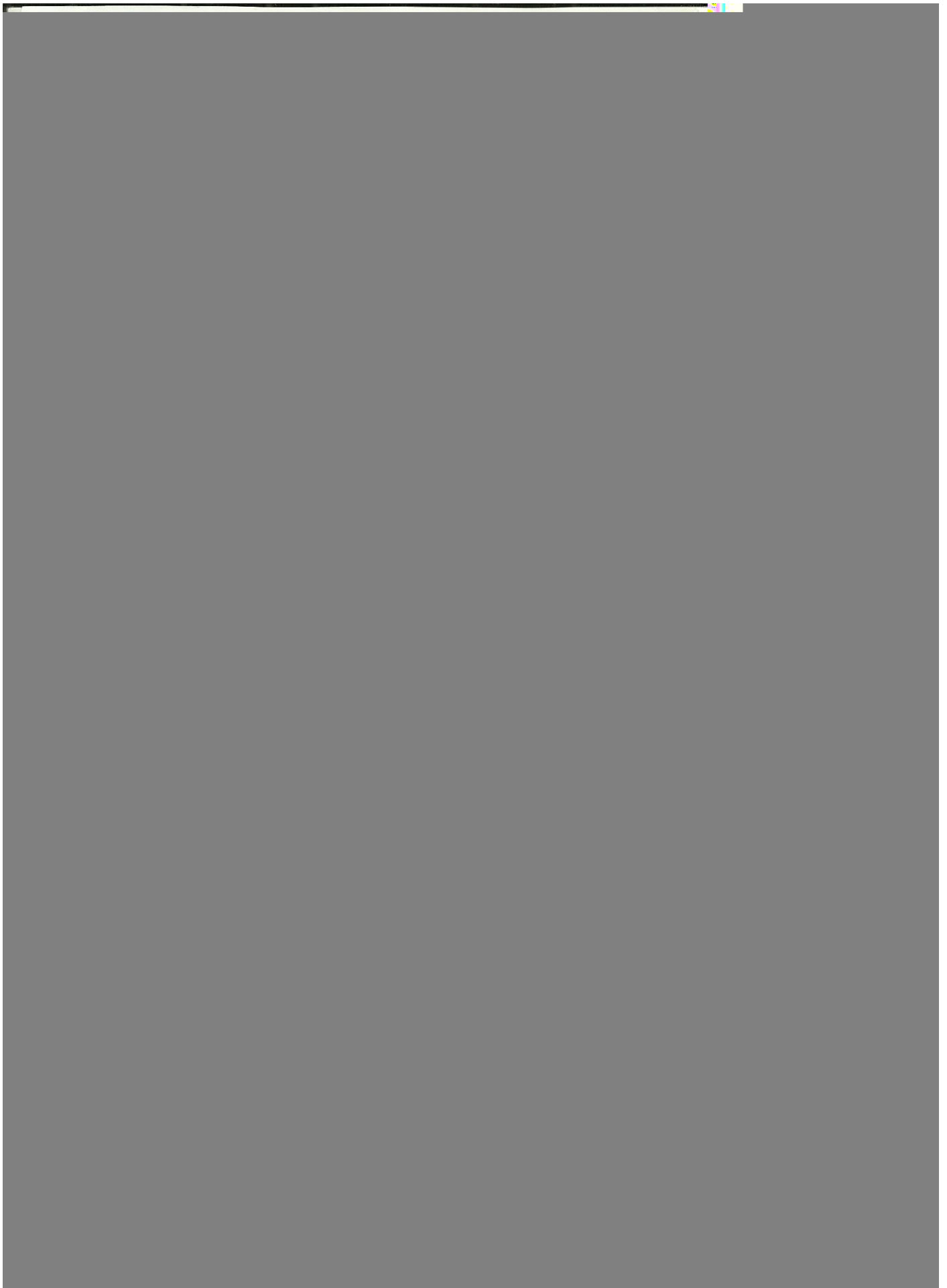
+ 3000 enregistres 3 fois

\$at heure [date] [+ increment]

> commande



UNIX



Les variétés

\$cours = Unix

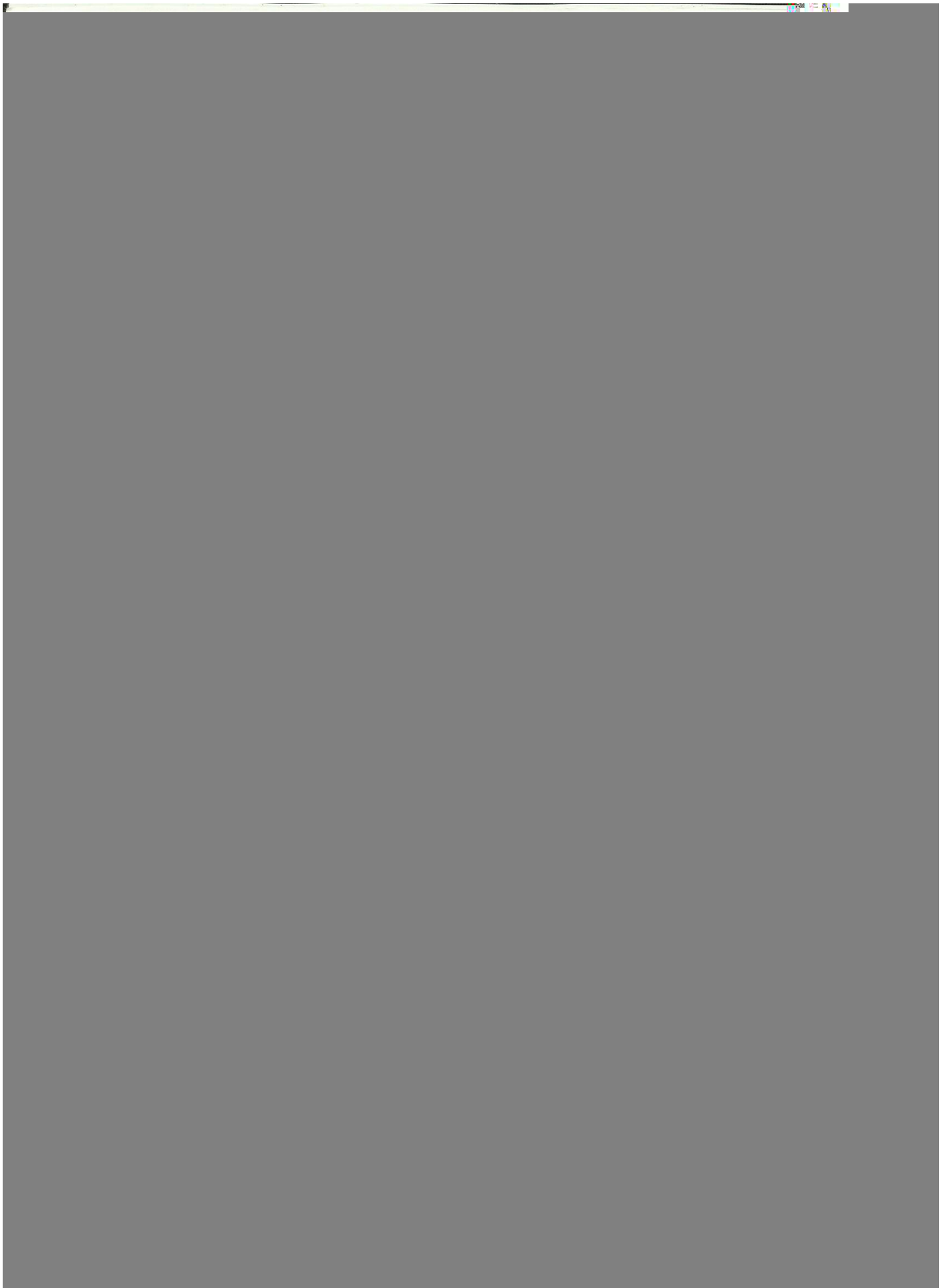
\$echo \$cours

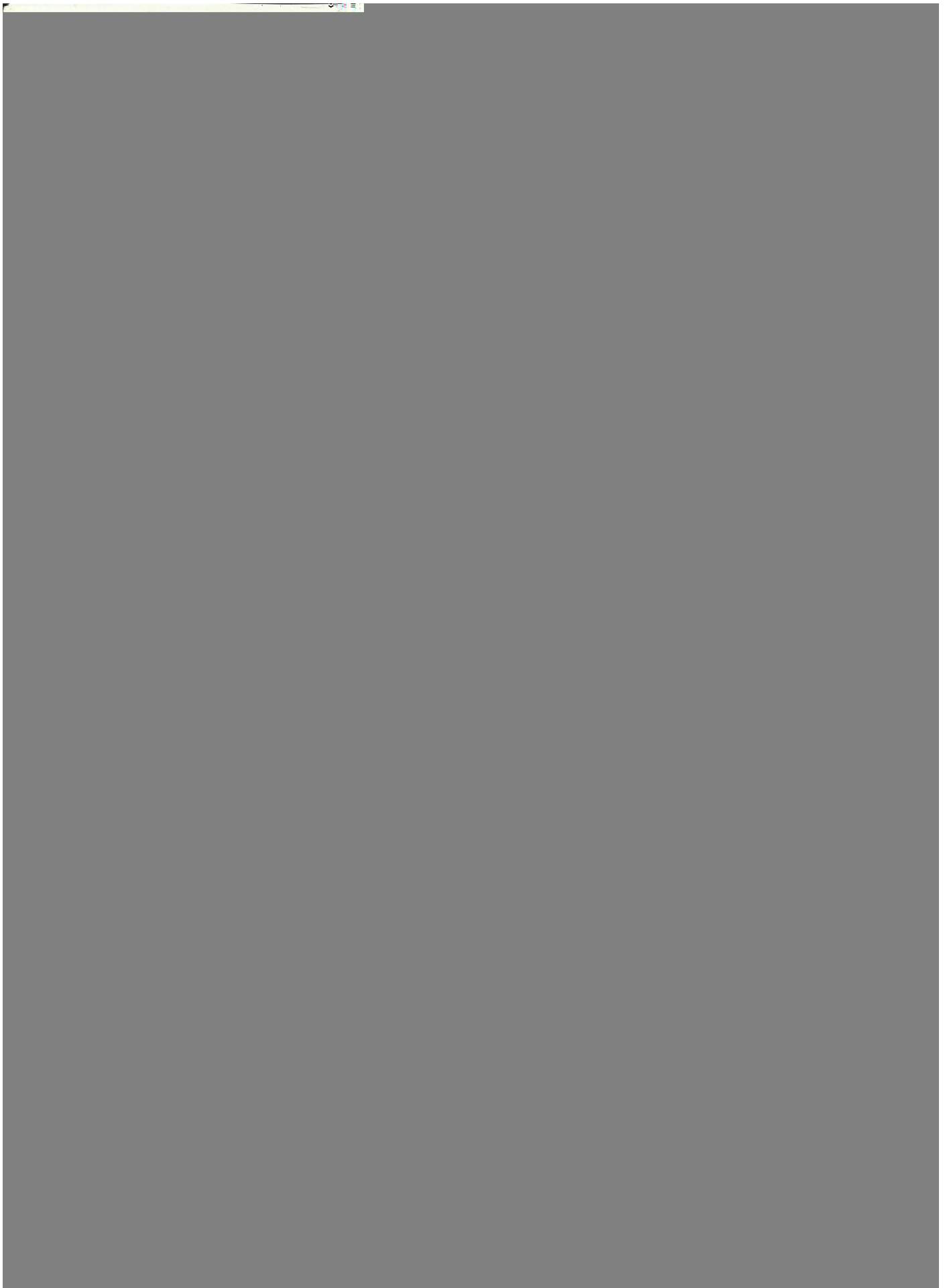
Unix

\$sh----->\$echo \$cours

Remarque :

- On peut lire une liste de variable





ne des paramètres positionnels

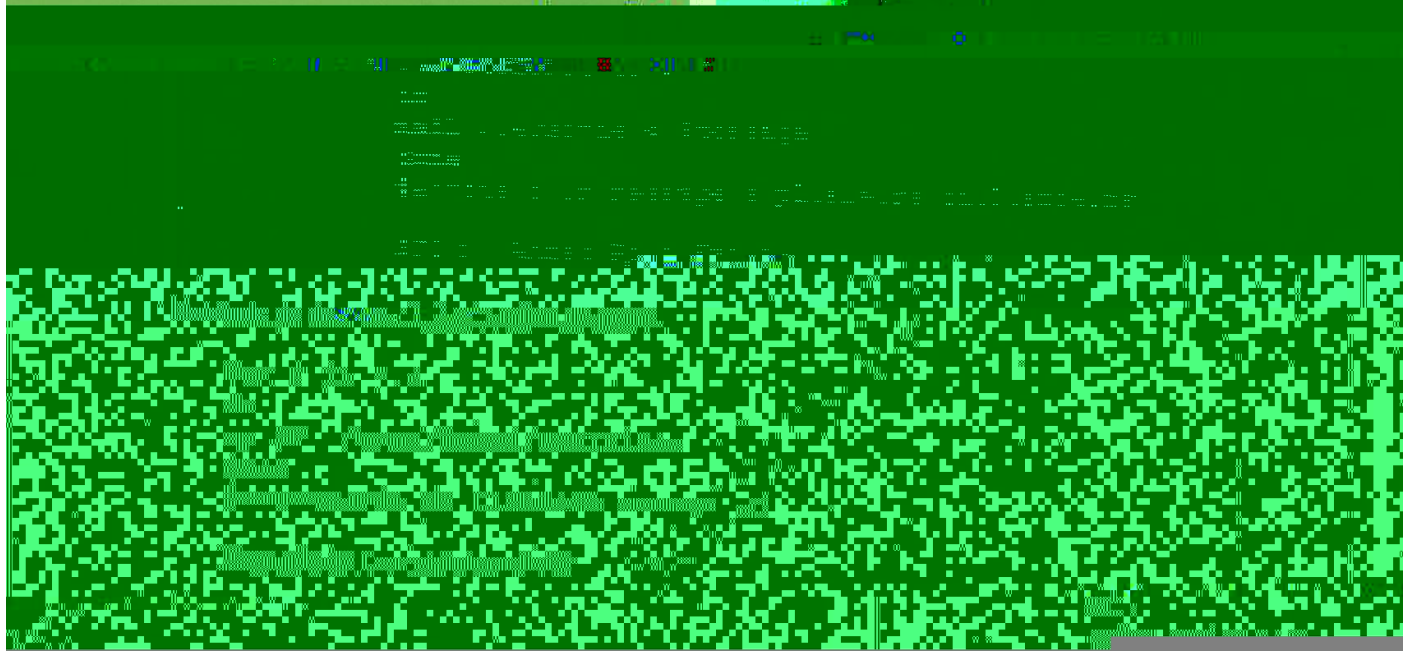
	A	B	C	D
shift		\$1	\$2	\$3

Exemple de for avec shift

shift effectue un décalage à gauche

A	B	C	D
\$1	\$2	\$3	\$4

SC :



Boucle while

Exemples :

SC1:

echo 'n° du programme que vous souhaitez? '

echo '1-date 2-who'

echo '3-ls 4-pwd'

read choix # entrée de la valeur

case \$choix in

1) date

1)

date —

5)

"1" "date"

Exemples :

SC :
destruction d'un fichier parmi deux identiques
usage : SC1 F1 F2
if cmp -s \$1 \$2 # -s : état de sortie de cmp
then
rm \$1; echo \$1 et \$2 sont identiques
else
echo \$1 et \$2 sont des fichiers distincts
fi

et affiche les lignes qui sont différentes

identiques : le contenu
pas le nom

On peut imbriquer des tests if de la manière suivante :

if commande_de_contrôle_1
then
commandes_A
elif commande_de_contrôle_2
then
commandes_B
else
commandes_C
fi

Si succès (exit 0) de commande_de_contrôle_1 alors
exécution des commandes_A

sinon

si succès (exit 0) de commande_de_contrôle_2 alors
exécution des commandes_B

Exemples :

```
SC1:
#deviner un nombre caché entre 1 et 50
echo Je pense a un nombre entre 1 et 50
echo Lequel \?
read reponse
until test $reponse=33
do
echo 'dommage, essayez encore!'
read reponse
done
echo 'Bien vu!'
```

```
$SC1
Je pense a un nombre entre 1 et 50
Lequel?
5
Dommage, essayez encore!'
33
Bien vu!
$
```

```
SC2:
# deviner le meilleur OS
if test $1=Unix      # $1 premier paramètre
then
    echo je suis d\'accord
else
    echo Je n\'ai jamais entendu parler de $1
fi
```

Test d'état de fichier

test -r F	#vrai, si F existe avec le droit en lecture	le contraire : !-r F
-w F	#vrai, si F existe avec le droit en ecriture	
-f F	#vrai, si F existe et n'est pas un répertoire	
-d F	#vrai, si F existe et c'est un répertoire	
-s F	#vrai, si F existe et sa taille > 0	
-x F	#vrai, si F existe et c'est un exécutable	

Exemple :

```
$test -w .profile; echo $?
```

Test de comparaison de chaînes

```
test chaîne_1 = chaîne_2      # égalité
test chaîne_1 != chaîne_2     # non égalité
test -n chaîne                 # longueur non égal à 0  chaîne non vide
test -z chaîne                 # longueur 0  chaîne vide
test chaîne                    # chaîne inexistente ou vide
```

Test de comparaison numérique

```
test n1 -eq n2      # égalité      =
n1 -ne n2           # non égalité  !=
n1 -gt n2           # supérieur    >
n1 -lt n2           # inférieur    <
n1 -ge n2           # supérieur ou égal ≥
n1 -le n2           # inférieur ou égal ≤
```

Exemples :

```
test 5 = 5          # vrai comparaison de chaînes
test 5 = 05         # faux comparaison de chaînes
test 5 -eq 05       # vrai comparaison numérique.
```

test et opérations logiques

```
!          #négation
-a         #et logique  &&
-o         #ou logique  ||
```

Exemples :

```
test !-r F
#vrai, si F existe et si pas de droit en lecture

test -r F1 -a -w F2
# vrai, si droit en lecture sur le fichier F1
# et si droit en écriture sur le fichier F2

test -r F1 -o -w F1
# vrai, si droit en lecture ou en écriture sur F1
```

formes réduites :

[...] équivalent à `test`.

```
[ $nom = Ahmed ]      équivalent à      test $nom = Ahmed
[ -f fichier ]        équivalent à      test -f fichier
```

```
[ $age -gt 9 -a $age -le 20 ]
équivalent à test $age -gt 9 -a $age -le 20
```


Calcul arithmétique par expr

`expr` interprète et évalue une expression arithmétique; le résultat est produit en sortie.

Exemple

`$expr 3+5`
8
`$expr 9-2`
7
`$expr 4\*3`
12
`$expr 12/3`
4

il faut travailler avec des entiers !!

SC:

conversion minutes-secondes

case \$# in

0) echo pas d'argument\!; exit 1;;

*) min = \$1;;

esac

while test \$1

do

echo 'expr 60 * \$min'

shift; min = \$1

done

l'annule l'effet de *

Factorielle:

#calcul récursif

case \$1 in

1) echo 1;;

$$n! = n \cdot (n-1)!$$

$$f(n) = n \cdot f(n-1)$$

continue pour passer à l'itération suivante

Exemple :

SC :

echo "saisir le texte (q=quit, 'n=continue) "

while True

do

echo "nom: \c"